

Move-To-Front

El algoritmo **Move-To-Front (MTF)** es una heurística utilizada en estructuras de datos dinámicas para mejorar el acceso a elementos que se consultan frecuentemente. Su funcionamiento básico es el siguiente:

- Se mantiene una lista o arreglo con las llaves o elementos en un orden inicial arbitrario y desordenado.
- Para buscar un elemento, se realiza una búsqueda secuencial desde el frente de la lista hasta encontrarlo.
- Una vez encontrado el elemento, este se mueve inmediatamente al frente de la lista.

Este movimiento al frente busca aprovechar la *localidad temporal* de las consultas, es decir, la probabilidad de que un elemento recientemente accedido sea accedido nuevamente pronto. De esta forma, los elementos más solicitados tienden a permanecer en posiciones delanteras, reduciendo el costo promedio de acceso.

MTF es un algoritmo en línea, ya que toma decisiones basadas solo en la información de las consultas realizadas hasta el momento, sin conocer futuras solicitudes. Su desempeño se mide comparándolo con un algoritmo óptimo que conoce toda la secuencia de consultas de antemano (análisis competitivo).

MTF tiene un *competitive ratio* de 2, lo que significa que su costo total de acceso es a lo más el doble que el óptimo offline:

- Sleator y Tarjan [1] demostraron que el costo amortizado de acceder a una secuencia de elementos con MTF es a lo más el doble del costo óptimo offline, más una constante aditiva.
- Esto se debe a que MTF, al mover cada elemento accedido al frente, mantiene una lista que se adapta a las frecuencias y patrones de acceso, pero sin conocer el futuro, puede incurrir en un costo hasta 2 veces mayor que el óptimo.

En resumen, Move-To-Front es un método simple y efectivo para mantener una lista adaptativa que favorece el acceso rápido a elementos frecuentemente consultados, mediante la estrategia de mover al frente cada elemento accedido. MTF es útil en problemas donde el patrón de acceso a elementos tiene cierta repetición o sesgo hacia algunos elementos, mejorando la eficiencia respecto a búsquedas siempre en un arreglo estático.

Análisis del costo asintótico

Tiempo

- La búsqueda del símbolo en la lista es secuencial, por lo que en el peor caso es $O(m)$, donde m es el tamaño de la lista.
- Mover un elemento al frente es $O(m)$ en listas implementadas como arreglos (por el desplazamiento).
- Por lo tanto, cada operación de codificación o decodificación es $O(m)$ en el peor caso.
- Para una secuencia de n símbolos, el costo total es $O(m \times n)$.

Espacio

- Se mantiene una lista de tamaño m para los símbolos.
- El espacio adicional para la codificación es $O(n)$ para almacenar la secuencia codificada.
- Por tanto, el espacio es $O(n+m)$

Este análisis muestra que MTF es eficiente para alfabetos pequeños o cuando la localidad temporal reduce el costo promedio, pero puede ser costoso para alfabetos muy grandes sin optimizaciones.

Referencias

[1]. Sleator, D. D., & Tarjan, R. E. (1985). *Amortized Efficiency of List Update and Paging Rules*. Communications of the ACM, 28(2), 202–208.

Disponible en: <https://www.cs.cmu.edu/~sleator/papers/amortized-efficiency.pdf>